

Project Guide: Process Equipment Selection Assistant

1. Project Goal

This project is a local-first engineering software prototype for a hackathon-style AI agent system.

The main goal is **not necessarily to win the hackathon prize**, but to build a clean, runnable, end-to-end project from scratch. The priority is to make the complete workflow work locally first, then optionally wrap the working logic into Fetch.ai/uAgents/Agentverse/ASI:One later.

The project should be designed in a modular way so that each part of the logic can later become an agent, tool, module, or API endpoint.

2. Product Concept

The project is a **process equipment selection assistant**.

It takes simplified lab-scale reaction or process data as input and helps generate a preliminary process design workflow:

1. Understand the lab-scale process input.
2. Generate a preliminary process basis.
3. Decompose the process into major unit operations.
4. Select possible equipment types for each unit operation.
5. Screen equipment candidates using engineering constraints.
6. Provide recommended equipment choices with reasons.
7. Match the selected equipment with mock suppliers.
8. Estimate rough CAPEX ranges using mock data.
9. Generate an RFQ-style supplier contact draft.

The system is intended as an early-stage engineering decision-support tool, not a final design tool.

3. Important Scope Boundary

Please do not overengineer the first version.

The first version should be a **local logic-only MVP**. It does not need real web search, real supplier APIs, real ASPEN integration, real purchasing, or real Agentverse deployment.

Use mock data where needed.

The first goal is:

A user can run one command locally and get a complete sample output report from a sample input file.

For example:

```
python main.py --input data/sample_input.json
```

Expected output:

```
outputs/sample_report.md  
outputs/sample_output.json
```

4. What This Project Is Not

This project is not trying to produce a certified engineering design.

It should not claim to replace a professional process engineer.

It should not make guaranteed safety, cost, or procurement decisions.

It should not actually purchase equipment or contact real suppliers.

It should not rely on hidden API keys or online services for the MVP.

The correct positioning is:

preliminary process design support, equipment screening, and structured decision assistance.

5. Current Project Structure

Please keep the project clean and readable.

Current intended structure:

```
process-equipment-agent/  
  README.md  
  requirements.txt  
  .gitignore  
  main.py  
  data/  
    sample_input.json
```

```
equipment_database.json
supplier_database.json
src/
  process_basis.py
  unit_decomposer.py
  equipment_selector.py
  engineering_reviewer.py
  supplier_matcher.py
  rfq_generator.py
outputs/
  sample_report.md
  sample_output.json
docs/
  project_guide.md
  architecture.md
```

6. Module Responsibilities

main.py

The main entry point.

It should: 1. Load the sample input JSON. 2. Call each module in order. 3. Save final structured output to JSON. 4. Save a readable Markdown report.

Keep it simple and deterministic.

src/process_basis.py

Creates a preliminary process basis from the lab-scale input.

It may extract or infer:

- target product
- reactants
- reaction phase
- operating temperature
- operating pressure
- desired production scale
- safety concerns
- material compatibility concerns
- major assumptions
- missing information

This should not pretend to be a full process simulator.

src/unit_decomposer.py

Breaks the process into likely unit operations.

For MVP, support only a small number of unit types:

- reactor
- separator
- heat exchanger

It should return structured unit operation data.

src/equipment_selector.py

Uses the mock equipment database to find candidate equipment for each unit operation.

It should match based on:

- unit type
- phase
- temperature range
- pressure range
- material compatibility
- typical application
- risk notes

src/engineering_reviewer.py

Screens equipment candidates against process constraints.

It should explain:

- why each candidate is accepted
- why each candidate is rejected
- what assumptions are being made
- what risks remain

This module should prioritize transparency over complexity.

src/supplier_matcher.py

Uses mock supplier data to match recommended equipment with possible suppliers.

For MVP, supplier data should come from `data/supplier_database.json`.

No real web search is required in v1.

src/rfq_generator.py

Generates an RFQ-style draft based on final equipment recommendations.

The RFQ draft should include:

- equipment type
- required operating conditions
- material compatibility requirements
- approximate capacity
- questions for supplier
- request for quote wording

7. Data Files

data/sample_input.json

A single sample case used for the first demo.

It should contain simplified lab-scale data, for example:

- process name
- target product
- reactants
- reaction type
- phase
- lab-scale amount
- target scale
- operating temperature
- operating pressure
- known hazards
- material concerns
- separation needs
- heating/cooling needs

data/equipment_database.json

Mock database of equipment options.

Each equipment entry should include:

- equipment_type
- name
- suitable_phase
- temperature_range_c
- pressure_range_bar
- material_compatibility

- typical_applications
- pros
- cons
- risk_notes

data/supplier_database.json

Mock supplier database.

Each supplier entry should include:

- supplier_name
- equipment_type
- location
- estimated_price_range_usd
- lead_time
- notes

8. Desired MVP Workflow

The first runnable version should follow this sequence:

```

sample_input.json
  ↓
process_basis.py
  ↓
unit_decomposer.py
  ↓
equipment_selector.py
  ↓
engineering_reviewer.py
  ↓
supplier_matcher.py
  ↓
rfq_generator.py
  ↓
sample_output.json + sample_report.md

```

The output report should be human-readable and suitable for demo.

9. Engineering Philosophy

Keep the first version:

- simple

- deterministic
- readable
- modular
- easy to explain
- easy to test
- easy to later wrap into an agent

Avoid:

- complex frameworks too early
- unnecessary web apps
- unnecessary databases
- overcomplicated agent orchestration
- real purchasing flows
- hidden online dependencies
- fragile external API calls

10. Later Fetch.ai / Agent Extension

After the local MVP works, the project may be wrapped into a Fetch.ai/uAgents system.

Possible future architecture:

- Coordinator uAgent:
 - receives user request through ASI:One
 - calls the local workflow
- returns structured equipment recommendation
- Optional specialist agents:
 - Reactor Specialist Agent
 - Separator Specialist Agent
 - Heat Exchanger Specialist Agent
 - Supplier Matching Agent

However, this should come after the local workflow works.

The first priority is a working local product.

11. Current Development Instruction

Please help build the local MVP first.

Do not start with Agentverse, ASI:One, or deployment.

Do not implement real web search yet.

Do not implement real ASPEN integration yet.

Do not create a large frontend yet.

Please create a clean, runnable Python project that demonstrates the full workflow end-to-end with mock data.

The project should be understandable to a beginner and maintainable for future extension.